Below is an easy, beginner-friendly guide to fundamental Dart programming concepts with your requested topics:

1. Dart Programming Introduction

Dart is a programming language developed by Google. It is mainly used to build Flutter apps for Android, iOS, Web, Windows, macOS, and Linux.

A basic Dart program always starts from the main() function.

Dart

```
void main() {
  print("Hello Dart!");
}
```

Output:

Plain text

```
Hello Dart!
```

Roman Nepali Explanation

Dart programming ma main() function bata program start huncha. print() function le screen ma output dekhauncha. Yo C/C++ ko main() jastai ho.

2. Dart Variables and Data Types

Variables are used to store data.

Common Dart data types are:

Data Type	Use
int	Whole number
double	Decimal number
String	Text
bool	true/false

var

Automatically detects type

dynamic

Can change type later

Dart

```
void main() {
  int age = 21;
  double height = 5.8;
  String name = "Loknath";
  bool isStudent = true;

  var college = "Pokhara University";
  dynamic value = 100;

  value = "Now I am text";

  print(name);
  print(age);
  print(height);
  print(isStudent);
  print(college);
  print(value);
}
```

Roman Nepali Explanation

Variable vaneko data store garne container ho. int le number, double le decimal, String le text, bool le true/false value store garcha. var use garda Dart le type aafai detect garcha. dynamic ma chai pachi type change garna milcha.

3. Dart Operators

Operators are symbols used to perform operations.

Dart

```
void main() {
  int a = 10;
  int b = 5;

  print(a + b); // Addition
  print(a - b); // Subtraction
  print(a * b); // Multiplication
  print(a / b); // Division
  print(a % b); // Remainder
}
```

Output:

Plain text

```
15
5
50
2.0
0
```

Roman Nepali Explanation

Operator le calculation garna help garcha. + addition, - subtraction, * multiplication, / division, % remainder nikalna use huncha.

4. Conditional Statement: if-else

Condition check garna if-else use huncha.

Dart

```
void main() {
  int marks = 75;

  if (marks >= 80) {
```

```
    print("Distinction");
  } else if (marks >= 60) {
    print("First Division");
  } else if (marks >= 40) {
    print("Pass");
  } else {
    print("Fail");
  }
}
```

Output:

Plain text

First Division

Roman Nepali Explanation

if-else le condition check garcha. Condition true bhayo bhane tyo block run huncha. Mathi ko example ma marks 75 cha, tesaile First Division print huncha.

5. Loops in Dart

Loop is used to repeat code.

for loop

Dart

```
void main() {
  for (int i = 1; i <= 5; i++) {
    print("Number: $i");
  }
}
```

while loop

Dart

```
void main() {
  int i = 1;
```

```
  while (i <= 5) {
    print(i);
    i++;
  }
}
```

Roman Nepali Explanation

Loop le same code repeated run garna help garcha. for loop generally fixed number of times run garna use huncha. while loop condition true huda samma run huncha.

6. Functions in Dart

Function is a reusable block of code.

Dart

```
void greetUser() {
  print("Welcome to Dart Programming");
}

void main() {
  greetUser();
}
```

Function with parameter:

Dart

```
void addNumbers(int a, int b) {
  int sum = a + b;
  print("Sum is $sum");
}

void main() {
  addNumbers(10, 20);
```

```
}
```

Function with return value:

Dart

```
int multiply(int a, int b) {
  return a * b;
}


void main() {
  int result = multiply(5, 4);
  print("Result: $result");
}
```

Roman Nepali Explanation

Function vaneko reusable code block ho. Same code baar-baar lekhnu parena. Function lai call garepachi matra run huncha. Parameter use garera function bhitra value pathauna sakincha. return le value firta dincha.

7. List in Dart

List is used to store multiple values.

Dart

```
void main() {
  List<String> students = ["Ram", "Sita", "Hari"];


  print(students[0]);
  print(students[1]);


  students.add("Gita");


  print(students);
}
```

Output:

Plain text

Ram

Sita

[Ram, Sita, Hari, Gita]

Roman Nepali Explanation

List vaneko multiple data store garne collection ho. List ko index 0 bata start huncha. students[0] le first value dincha. add() le new value add garcha.

8. Map in Dart

Map stores data in key-value pair.

Dart

```
void main() {
  Map<String, String> student = {
    "name": "Loknath",
    "college": "MBCOE",
    "faculty": "Computer Engineering"
  };

  print(student["name"]);
  print(student["college"]);
}
```

Roman Nepali Explanation

Map ma data key-value pair ma store huncha. Example: "name" key ho ra "Loknath" value ho. Map student ko details store garna useful huncha.

9. Null Safety in Dart

Null safety protects your program from null errors.

Dart

```
void main() {
  String name = "Loknath";
  String? address;
```

```
  print(name);
  print(address);
}
```

Here, String? means this variable can contain null.

Dart

```
void main() {
  String? name;

  print(name ?? "No name available");
}
```

Output:

Plain text

```
No name available
```

Roman Nepali Explanation

Dart ma null safety le variable null huda error aauna bata bachaucha. String name ma null rakhna mildaina, tara String? name ma null rakhna milcha. ?? le null bhayo bhane alternative value dincha.

10. OOP Concepts in Dart

OOP means Object-Oriented Programming. It helps to organize code using classes and objects.

Main OOP concepts are:

Class

Object

Constructor

Encapsulation

Inheritance

Polymorphism

Abstraction

10.1 Class and Object

Class is a blueprint. Object is a real instance of class.

Dart

```
class Student {
  String name = "Loknath";
  int age = 21;

  void displayInfo() {
    print("Name: $name");
    print("Age: $age");
  }
}

void main() {
  Student s1 = Student();
  s1.displayInfo();
}
```

Output:

Plain text

```
Name: Loknath
Age: 21
```

Roman Nepali Explanation

Class vaneko blueprint ho. Object vaneko class bata baneko real item ho. Mathi Student class ho ra s1 object ho. Object use garera class ko variable ra function access garna sakincha.

10.2 Constructor

Constructor automatically runs when object is created.

Dart

```
class Student {
```

```
 String name;
 int age;

 Student(this.name, this.age);

 void displayInfo() {
  print("Name: $name");
  print("Age: $age");
 }
}

void main() {
 Student s1 = Student("Loknath", 21);
 s1.displayInfo();
}
```

Roman Nepali Explanation

Constructor object create hune bittikai automatic run huncha. Constructor ko help le object create garda nai value initialize garna sakincha. Student(this.name, this.age) le short way ma value assign garcha.

10.3 Encapsulation

Encapsulation means hiding data and controlling access.

In Dart, private variables start with _.

Dart

```
class BankAccount {
 double _balance = 0;

 void deposit(double amount) {
  if (amount > 0) {
   _balance += amount;
```

```
  }
 }

 double getBalance() {
  return _balance;
 }
}

void main() {
 BankAccount account = BankAccount();

 account.deposit(5000);
 print("Balance: ${account.getBalance()}");
}
```

Roman Nepali Explanation

Encapsulation vaneko data hide garera secure rakhnu ho. _balance private variable ho. Direct access nagari method through access garna dincha. Bank account jasto sensitive data ma yo concept useful huncha.

10.4 Inheritance

Inheritance means one class can use properties and methods of another class.

Dart

```
class Animal {
 void eat() {
  print("Animal is eating");
 }
}

class Dog extends Animal {
 void bark() {
```

```
    print("Dog is barking");
  }
}

void main() {
  Dog dog = Dog();

  dog.eat();
  dog.bark();
}
```

Output:

Plain text

```
Animal is eating
Dog is barking
```

Roman Nepali Explanation

Inheritance ma child class le parent class ko property ra method use garna sakcha. Mathi Dog child class ho ra Animal parent class ho. extends keyword inheritance ko lagi use huncha.

10.5 Polymorphism

Polymorphism means one method can behave differently in different classes.

Dart

```
class Animal {
  void sound() {
    print("Animal makes sound");
  }
}

class Dog extends Animal {
  @override
```

```
  void sound() {
    print("Dog barks");
  }
}

class Cat extends Animal {
  @override
  void sound() {
    print("Cat meows");
  }
}

void main() {
  Animal a1 = Dog();
  Animal a2 = Cat();

  a1.sound();
  a2.sound();
}
```

Output:

Plain text

Dog barks

Cat meows

Roman Nepali Explanation

Polymorphism vaneko same method different class ma different behavior dinu ho. sound() method Animal, Dog, Cat ma different way ma kaam gareko cha. Yo OOP ko powerful concept ho.

10.6 Abstraction

Abstraction means hiding internal details and showing only important features.

Dart

```
abstract class Shape {
  void area();
}

class Circle extends Shape {
  double radius;

  Circle(this.radius);

  @override
  void area() {
    print("Area of circle: ${3.14 * radius * radius}");
  }
}

void main() {
  Circle c = Circle(5);
  c.area();
}
```

Roman Nepali Explanation

Abstraction vaneko unnecessary internal detail hide garera important functionality matra dekhaunu ho. abstract class ko object directly banauna mildaina. Child class le abstract method implement garnu parcha.

11. Override in Dart

@override is used when a child class changes the method of parent class.

Dart

```
class Parent {
  void message() {
```

```
   print("This is parent message");
  }
}

class Child extends Parent {
  @override
  void message() {
   print("This is child message");
  }
}

void main() {
  Child obj = Child();
  obj.message();
}
```

Output:

Plain text

This is child message

Roman Nepali Explanation

@override ko meaning ho parent class ko method lai child class ma change garera use garnu. Parent class ma message() thiyo, child class le same method lai aafno way ma define gareko cha.

12. Async in Dart

async is used when a function performs work that may take time, like:

API call

Database operation

File loading

Internet request

Timer delay

An async function usually returns a Future.

Dart

```
Future<void> fetchData() async {
  print("Fetching data...");
  await Future.delayed(Duration(seconds: 2));
  print("Data loaded successfully");
}

void main() {
  fetchData();
  print("Program continues...");
}
```

Output:

Plain text

```
Fetching data...
Program continues...
Data loaded successfully
```

Roman Nepali Explanation

async function time lagne kaam ko lagi use huncha. Example: internet bata data load garna, database bata data lina, file read garna. async function background jasto kaam garcha ra program block hudaina.

13. Await in Dart

await waits for a Future to complete.

Important rule:

Dart

```
await
```

can be used only inside an async function.

Dart

```
Future<String> getUserName() async {
```

```
  await Future.delayed(Duration(seconds: 2));
  return "Loknath Regmi";
}

void main() async {
  print("Getting user name...");

  String name = await getUserName();

  print("User name is $name");
}
```

Output:

Plain text

```
Getting user name...
User name is Loknath Regmi
```

Roman Nepali Explanation

await le Future complete hune bela samma wait garcha. Suppose internet bata username aauna 2 seconds lagcha, await le data aaunjel wait garcha ani pachi result use garcha. await use garna function async hunai parcha.

14. Async and Await Together

async and await usually work together.

Dart

```
Future<int> getMarks() async {
  await Future.delayed(Duration(seconds: 1));
  return 85;
}

void main() async {
  print("Checking marks...");
```

```
  int marks = await getMarks();

  if (marks >= 80) {
    print("Excellent");
  } else {
    print("Keep practicing");
  }
}
```

Roman Nepali Explanation

async le function asynchronous banaucha. await le result aauna wait garcha. Flutter app ma API bata data fetch garna yo concept dherai important huncha.

15. Future in Dart

A Future represents a value that will be available later.

Dart

```
Future<String> downloadFile() {
  return Future.delayed(Duration(seconds: 3), () {
    return "File downloaded";
  });
}

void main() async {
  print("Download started");

  String result = await downloadFile();

  print(result);
}
```

Roman Nepali Explanation

Future vaneko future ma result aaune promise jasto ho. Ahile result chaina, tara kehi time pachi result aaucha. API, database, file download jasto kaam ma Future use huncha.

---

16. Exception Handling in Dart

Error handle garna try-catch use garincha.

Dart

```
void main() {
  try {
    int result = 10 ~/ 0;
    print(result);
  } catch (e) {
    print("Error occurred: $e");
  }
}
```

Roman Nepali Explanation

Program run huda error aauna sakcha. Error aayo bhane program crash nahos bhanera try-catch use garincha. try bhitra risky code rakhincha, catch le error handle garcha.

17. Flutter Stateless Widget

A StatelessWidget is a widget that does not change its state after creation.

Example:

Dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
```

```
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: HomePage(),
    );
  }
}

class HomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text("Stateless Widget Example"),
      ),
      body: Center(
        child: Text(
          "Hello, I am Stateless Widget",
          style: TextStyle(fontSize: 22),
        ),
      ),
    );
  }
}
```

Roman Nepali Explanation

StatelessWidget tyo widget ho jasle UI change gardaina. Ekchoti build bhayepachi data change hudaina. Static text, icon, image, title, simple screen ko lagi StatelessWidget use garincha.

Example: About page, splash screen, simple profile card.

18. Flutter Stateful Widget

A StatefulWidget is a widget that can change its state.

Example: Counter app

Dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: CounterPage(),
    );
  }
}

class CounterPage extends StatefulWidget {
  @override
  State<CounterPage> createState() => _CounterPageState();
}

class _CounterPageState extends State<CounterPage> {
  int count = 0;

  void increaseCounter() {
```

```
  setState(() {
    count++;
  });
 }

 @override
 Widget build(BuildContext context) {
  return Scaffold(
   appBar: AppBar(
    title: Text("Stateful Widget Example"),
   ),
   body: Center(
    child: Text(
     "Count: $count",
     style: TextStyle(fontSize: 25),
    ),
   ),
   floatingActionButton: FloatingActionButton(
    onPressed: increaseCounter,
    child: Icon(Icons.add),
   ),
  );
 }
}
```

Roman Nepali Explanation

StatefulWidget tyo widget ho jasle UI change garna sakcha. Counter increase huda UI update huncha. setState() call garepachi Flutter le screen update garcha.

Example: counter app, login form, checkbox, radio button, dark mode toggle, quiz app question change.

19. Difference Between StatelessWidget and StatefulWidget

Basis

StatelessWidget

StatefulWidget

Meaning

State change hudaina

State change huncha

UI Update

Static UI

Dynamic UI

setState()

Use hudaina

Use huncha

Example

Text, icon, static page

Counter, form, quiz

Performance

Lightweight

Slightly heavier

Use Case

Fixed data

Changing data

Roman Nepali Explanation

StatelessWidget fixed UI ko lagi ho. StatefulWidget changing UI ko lagi ho. Button click garda value change huncha bhane StatefulWidget use garne. UI static cha bhane StatelessWidget use garne.

20. Complete Dart OOP Program Example

Dart

```
class Person {
```

```
  String name;
  int age;

  Person(this.name, this.age);

  void displayInfo() {
    print("Name: $name");
    print("Age: $age");
  }
}

class Student extends Person {
  String college;
  String faculty;

  Student(String name, int age, this.college, this.faculty)
      : super(name, age);

  @override
  void displayInfo() {
    print("Student Name: $name");
    print("Age: $age");
    print("College: $college");
    print("Faculty: $faculty");
  }
}

void main() {
```

```
 Student student = Student(
   "Loknath Regmi",
   21,
   "Madan Bhandari College of Engineering",
   "Computer Engineering",
 );

 student.displayInfo();
}
```

Roman Nepali Explanation

Yo program ma Person parent class ho. Student child class ho. Student le Person ko property inherit gareko cha. super(name, age) le parent class ko constructor call garcha. @override le parent ko displayInfo() method lai child class ma change gareko cha.

21. Complete Async/Await Program Example

Dart

```
Future<String> loginUser(String username, String password) async {
 print("Checking user login...");

 await Future.delayed(Duration(seconds: 2));

 if (username == "admin" && password == "1234") {
   return "Login Successful";
 } else {
   return "Invalid username or password";
 }
}

void main() async {
```

```
  String result = await loginUser("admin", "1234");


  print(result);
}
```

Output:

Plain text

Checking user login...

Login Successful

Roman Nepali Explanation

Yo example ma login process simulate gareko cha. Real app ma server bata login check garna time lagcha. Tesaile async-await use garincha. await Future.delayed() le 2 seconds wait gareko cha. Username/password match bhayo bhane login successful huncha.

22. Important Dart Keywords

Keyword

Meaning

void

Function does not return value

main()

Starting point of program

class

Create class

extends

Inheritance

super

Call parent class

this

Current object

@override

Redefine parent method

async

Makes function asynchronous

await

Waits for Future result

Future

Value available later

final

Value assigned once

const

Compile-time constant

var

Auto type detection

dynamic

Type can change

Roman Nepali Explanation

Dart ma keywords ko fixed meaning huncha. class, extends, async, await, Future, override jस्ता keywords Dart programming ma dherai important huncha. Flutter sikna ko lagi iniharu majjale bujhnu parcha.

23. Final Simple Revision

Concept

Easy Meaning

Dart

Flutter app banaune programming language

Variable

Data store garne place

Function

Reusable code block

Class

Blueprint

Object

Class bata baneko real item

Constructor

Object create huda automatic run huncha

Inheritance

Parent class ko feature child class le use garne

Override

Parent method lai child ma change garne

Async

Time lagne task handle garne

Await

Result aaunjel wait garne

StatelessWidget

UI change hudaina

StatefulWidget

UI change huncha

Roman Nepali Explanation

Flutter developer banna Dart ko basic concept strong hunu parcha. Sabai bhanda pahila variable, function, class-object bujhne. Tespachi OOP concepts, async-await, Future bujhne. Ani Flutter ma StatelessWidget ra StatefulWidget bujhyo bhane app development easy huncha.